```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

# Blastocyst Frame Prediction with Guided Diffusion

## Install Necessary Python Packages

```
!pip -q install lpips diffusers torchmetrics[image]
```

## Import Necessary Libraries

```
import os, random, math, itertools, time, json, shutil, re, gc
from pathlib import Path
from collections import defaultdict

import numpy as np
import pandas as pd
from PIL import Image
from PIL import ImageDraw, ImageFont
from tqdm.auto import tqdm

import torch, torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as T
import torchvision.transforms.v2 as v2
import torchvision.models as models
import torchvision.utils as vutils

from diffusers import UNet2DModel, DDPMScheduler, DPMSolverMultistepScheduler, EMAModel
import lpips
from torchmetrics.image.fid import FrechetInceptionDistance
import matplotlib.pyplot as plt

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed(SEED); torch.cuda.manual_seed_all(SEED)

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print("Running on", DEVICE)
```

```
Running on cuda
```

```
ROOT         = Path("/content/drive/MyDrive/Msc in AI/AI Applications/Human embryo time-lapse video dataset")
IMG_DIR      = ROOT/"embryo_dataset"
ANNOT_DIR    = ROOT/"embryo_dataset_annotations"
MODEL_DIR    = ROOT/"models"

# pretrained classifiers ----------------------------------------------------
PRESENCE_MODEL_PATH = MODEL_DIR/"presence_classifier.pth"   # ResNet-18 weights
PHASE_MODEL_PATH    = MODEL_DIR/"phase_classifier.pth"      # EfficientNetV2-S weights

assert IMG_DIR.is_dir(),     f"{IMG_DIR} not found"
assert ANNOT_DIR.is_dir(),   f"{ANNOT_DIR} not found"
assert PRESENCE_MODEL_PATH.is_file(), f"presence model missing"
assert PHASE_MODEL_PATH.is_file(),    f"phase model missing"

# morphokinetic phases ------------------------------------------------------
PHASES = [
    "tPB2","tPNa","tPNf",
    "t2","t3","t4","t5","t6","t7","t8","t9+",
    "tM","tSB","tB","tEB","tHB"
]
PHASE2IDX = {p:i for i,p in enumerate(PHASES)}
```

```
IDX2PHASE = {i:p for p,i in PHASE2IDX.items()}
NUM_PHASES = len(PHASES)
```

## Define Output Directories and Data Paths

```
# ----- Embryo-presence (binary) -------------------------------------------
presence_model = models.resnet18(weights=None)
presence_model.conv1 = nn.Conv2d(1,64,kernel_size=7,stride=2,padding=3,bias=False)
presence_model.fc    = nn.Linear(presence_model.fc.in_features, 2)
presence_model.load_state_dict(torch.load(PRESENCE_MODEL_PATH, map_location="cpu"))
presence_model.to(DEVICE).eval().requires_grad_(False)

PRESENCE_SIZE = 224
presence_tfm = T.Compose([
    T.Resize((PRESENCE_SIZE, PRESENCE_SIZE)),
    T.ToTensor(),
    T.Normalize([0.5],[0.5]),
])

@torch.no_grad()
def embryo_prob(pil_img: Image.Image):
    """P(embryo present) from PIL image (grayscale)."""
    x = presence_tfm(pil_img).unsqueeze(0).to(DEVICE)
    logits = presence_model(x)
    return logits.softmax(1)[0,1].item()

# ----- Phase classifier ---------------------------------------------------
phase_model = models.efficientnet_v2_s(weights=None)
# adapt to grayscale -------------------------------------------------------
phase_model.features[0][0] = nn.Conv2d(1, phase_model.features[0][0].out_channels, kernel_size=3,stride=2,padding=1,bias=False)
phase_model.classifier[1]  = nn.Linear(phase_model.classifier[1].in_features, NUM_PHASES)
phase_model.load_state_dict(torch.load(PHASE_MODEL_PATH, map_location="cpu"))
phase_model.to(DEVICE).eval().requires_grad_(False)

PHASE_IMG_SIZE = 500            # ← unchanged
phase_tfm = T.Compose([
    T.CenterCrop(PHASE_IMG_SIZE),      # keeps 500×500 if already that size
    T.ToTensor(),
    T.Normalize([0.5],[0.5]),
])
```

## Get Valid Embryos

```
def index_frames(folder: Path):
    """Return {frame_idx: path} mapping for *_RUN###.* files."""
    mapping = {}
    for fp in folder.glob("*_RUN*.*"):
        m = re.search(r"_RUN(\d+)", fp.name)
        if m:
            mapping[int(m.group(1))] = fp
    return mapping

@torch.no_grad()
def keep_embryo_frames(paths, thr=0.5):
    """Filter list[Path] keeping only those with P(embryo) ≥ thr."""
    if not paths: return []
    tensors, kept = [], []
    for p in paths:
        try:
            img = Image.open(p).convert("L")
        except Exception:
            continue
        tensors.append(presence_tfm(img))
        kept.append(p)
    if not tensors:
        return []
    batch = torch.stack(tensors).to(DEVICE)
    probs = presence_model(batch).softmax(1)[:,1]
    return [p for p,pr in zip(kept, probs) if pr >= thr]
```

## Dataset Class

```
# ------------------------------------------------------------
# fast subsampling per phase
FRAMES_PER_PHASE = 5

# %%
# ------------------------------------------------------------
# 5. Build meta-index of *usable* frames for every embryo
# ------------------------------------------------------------
MAX_EMBRYOS = 150  # set to 10 for quick debug

VALID_EMBRYOS = []
print("Indexing embryos …")
for csv in tqdm(sorted(ANNOT_DIR.glob("*_phases.csv"))):
    eid = csv.stem.replace("_phases","")
    folder = IMG_DIR/eid
    if not folder.is_dir():
        continue

    # original mapping (all frames on disk)
    idx2path_raw = index_frames(folder)
    if len(idx2path_raw) == 0:
        continue

    phases = pd.read_csv(csv, header=None, names=["phase","start","end"])

    # subsample + presence-filter ----------------------------------------
    idx2path = {}
    for phase, start, end in phases.itertuples(index=False):
        if phase not in PHASE2IDX: continue
        rng = np.linspace(start, end, FRAMES_PER_PHASE, dtype=int) if (end-start+1) > FRAMES_PER_PHASE else range(start,end+1)
        cand_paths = [idx2path_raw.get(i) for i in rng if idx2path_raw.get(i)]
        kept = keep_embryo_frames(cand_paths, thr=0.5)
        for p in kept:
            m = re.search(r"_RUN(\d+)", p.name); idx = int(m.group(1))
            idx2path[idx] = p

    if len(idx2path) < 3:
        continue   # skip embryo with too few valid frames

    VALID_EMBRYOS.append({
        "id"        : eid,
        "idx2path"  : idx2path,
        "sorted_idx": sorted(idx2path.keys()),
        "phases"    : [(row.phase.strip(), int(row.start), int(row.end)) for row in phases.itertuples(index=False)]
    })
    if MAX_EMBRYOS and len(VALID_EMBRYOS) >= MAX_EMBRYOS:
        break
print(f"✓ {len(VALID_EMBRYOS)} embryos ready (filtered & subsampled).")
```

```
Indexing embryos …
 22%                                           152/704 [03:03<11:29,  1.25s/it]
✓ 150 embryos ready (filtered & subsampled).
```

## Image Preprocessing and Augmentation

```
class PhaseProgressPairs(Dataset):
    """Return dict{cond, target, progress, phase_tgt_idx}.
    cond    : random frame inside phase k
    target  : frame inside phase k+1 at relative progress α
    progress: α ∈ [0,1]
    phase_tgt_idx : *index* of phase k+1 (guidance label)
    """
    def __init__(self, embryo_meta, split="train", val_frac=0.1, IMG_SIZE=256, seed=SEED):
        self.IMG_SIZE = IMG_SIZE
        base_tfm = T.Compose([
          T.CenterCrop(IMG_SIZE),             # keeps 500×500 (never up-scales)
          T.Pad(PAD, fill=-1),
          T.ToTensor(),
          T.Normalize([0.5],[0.5]),
        ])
```

```
            aug = v2.Compose([
                v2.RandomHorizontalFlip(),
                v2.RandomVerticalFlip(),
                v2.RandomRotation(20),
            ])
            self.tfm = base_tfm if split!="train" else v2.Compose([aug, *base_tfm.transforms])
            rng = random.Random(seed); rng.shuffle(embryo_meta)
            cut = int(len(embryo_meta)*(1-val_frac))
            self.meta = embryo_meta[:cut] if split=="train" else embryo_meta[cut:]

        # -------------- helpers ------------------------------------------------
        def _phase_indices(self, s,e, idx2path):
            return [i for i in range(s,e+1) if i in idx2path]

        def _load(self, p):
            img = Image.open(p).convert("L")
            return self.tfm(img)

        def __getitem__(self, _):
            while True:
                emb = random.choice(self.meta)
                idx2p, phases = emb["idx2path"], emb["phases"]
                valid = [k for k in range(len(phases)-1)
                         if self._phase_indices(phases[k][1],phases[k][2],idx2p)
                         and self._phase_indices(phases[k+1][1],phases[k+1][2],idx2p)]
                if not valid: continue
                k = random.choice(valid)
                # ---- sample frames ----------------------------------------
                s0,e0 = phases[k][1:]; cond_idx = random.choice(self._phase_indices(s0,e0,idx2p))
                s1,e1 = phases[k+1][1:]
                α = random.random(); pos = s1 + int(α*(e1-s1))
                tgt_idx_choices = [j for j in (pos,pos-1,pos+1,s1,e1) if j in idx2p];
                if not tgt_idx_choices: continue
                tgt_idx = random.choice(tgt_idx_choices)
                cond = self._load(idx2p[cond_idx])
                tgt  = self._load(idx2p[tgt_idx])
                return {
                    "cond"    : cond,                   # [1,H,W]
                    "target"  : tgt,                    # [1,H,W]
                    "progress": torch.tensor([α],dtype=torch.float32),
                    "phase_tgt_idx": torch.tensor(PHASE2IDX[phases[k+1][0]],dtype=torch.long)
                }

        def __len__(self):
            return len(self.meta)*16   # heuristic

    # build loaders ------------------------------------------------------------
    IMG_SIZE   = 500          # true content
    UNET_SIZE  = 512          # divisible by 2**4 → avoids 125 vs 126 mismatch
    PAD        = (UNET_SIZE - IMG_SIZE) // 2     # = 6 px per side
    BATCH    = 4
    NUM_WORKERS = 2

    ds_train = PhaseProgressPairs(VALID_EMBRYOS, split='train', IMG_SIZE=IMG_SIZE)
    ds_val   = PhaseProgressPairs(VALID_EMBRYOS, split='val',   IMG_SIZE=IMG_SIZE)

    dl_train = DataLoader(ds_train, batch_size=BATCH, shuffle=True,  num_workers=NUM_WORKERS, pin_memory=True)
    dl_val   = DataLoader(ds_val,   batch_size=BATCH, shuffle=False, num_workers=NUM_WORKERS, pin_memory=True)

    print("Train batches", len(dl_train), "| Val batches", len(dl_val))
```

```
Train batches 540 | Val batches 60
```

## Setup Data Loaders

```
IN_CH = 1 + 1 + 1 + NUM_PHASES  # = 19
unet = UNet2DModel(
    sample_size        = UNET_SIZE,
    in_channels        = IN_CH,
    out_channels       = 1,
    layers_per_block   = 2,
    block_out_channels = [32, 64, 128, 128], # Reduced block out channels
    down_block_types   = ("DownBlock2D","AttnDownBlock2D","AttnDownBlock2D","AttnDownBlock2D"),
    up_block_types     = ("AttnUpBlock2D","AttnUpBlock2D","AttnUpBlock2D","UpBlock2D"),
    norm_eps = 1e-5,
```

```
)

noise_scheduler = DDPMScheduler(num_train_timesteps=1000)
dpm_solver      = DPMSolverMultistepScheduler.from_config(noise_scheduler.config)
ema             = EMAModel(unet.parameters(), decay=0.999)

unet.to(DEVICE); ema.to(DEVICE)

# optimiser ------------------------------------------------------------------
optimizer = torch.optim.AdamW(unet.parameters(), lr=1e-4, weight_decay=1e-4)
scaler    = torch.cuda.amp.GradScaler()

# FID setup ------------------------------------------------------------------
fid = FrechetInceptionDistance(feature=64, reset_real_features=False).to(DEVICE)
with torch.no_grad():
    for vb in tqdm(dl_val, desc="FID real cache"):
        fid.update(((vb["target"]+1)*127.5).clamp(0,255).to(torch.uint8).expand(-1,3,-1,-1).to(DEVICE), real=True)

lpips_loss = lpips.LPIPS(net='alex').to(DEVICE)

def one_hot_phase(idx, H,W):
    """ idx: [B] long → [B,C,H,W] one-hot spatial map """
    B = idx.size(0)
    oh = F.one_hot(idx, NUM_PHASES).float().to(DEVICE)           # [B,C]
    return oh.view(B,NUM_PHASES,1,1).expand(-1,-1,H,W)
```

```
FID real cache: 100%                                                60/60 [00:01<00:00, 44.35it/s]
Setting up [LPIPS] perceptual loss: trunk [alex], v[0.1], spatial [off]
/usr/local/lib/python3.11/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated sin
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None`
  warnings.warn(msg)
Loading model from: /usr/local/lib/python3.11/dist-packages/lpips/weights/v0.1/alex.pth
```

## Setting up the Diffusion Model and Related Components

```
@torch.no_grad()
def ssim_batch(x,y, eps=1e-2):
    mu_x, mu_y = x.mean([2,3]), y.mean([2,3])
    s_x,  s_y  = (x**2).mean([2,3]), (y**2).mean([2,3])
    return (((2*mu_x*mu_y+eps)/(s_x+s_y+eps)).clamp(0,1)).mean()

@torch.no_grad()
def sample_phase(cond, progress, phase_idx, steps=20):
    """Plain (unguided) sampling with *known* target phase index."""
    ema.store(unet.parameters()); ema.copy_to(unet.parameters())
    B,_,H,W = cond.shape
    prog_map  = progress.view(B,1,1,1).expand(-1,-1,H,W)
    phase_map = one_hot_phase(phase_idx, H,W)
    dpm_solver.set_timesteps(steps)
    x = torch.randn_like(cond)
    for t in dpm_solver.timesteps:
        noise_pred = unet(torch.cat([x, cond, prog_map, phase_map],1), t).sample
        x = dpm_solver.step(noise_pred, t, x).prev_sample
    ema.restore(unet.parameters())
    return x.clamp(-1,1)
```

## Setup Frechet Inception Distance (FID)

```
def sample_phase_guided(cond, progress, phase_idx, steps=20, guidance_scale=400):
    """Classifier guidance (Dhariwal & Nichol 2021) using the *phase classifier*.
    guidance_scale ~ σ in pixels – tune between 100 and 1000.
    """
    ema.store(unet.parameters()); ema.copy_to(unet.parameters())
    B,_,H,W = cond.shape
    prog_map  = progress.view(B,1,1,1).expand(-1,-1,H,W)
    phase_map = one_hot_phase(phase_idx, H,W)
    target_phase = phase_idx

    dpm_solver.set_timesteps(steps)
    x = torch.randn_like(cond)
```

```
        for t in dpm_solver.timesteps:
            # ----- classifier grad ----------------------------------------
            x_in = x.detach().requires_grad_(True)
            # resize to 500×500 for phase_model
            inp = x_in[:, :, PAD:-PAD, PAD:-PAD]
            logits = phase_model(inp)
            log_prob = F.log_softmax(logits,1)[torch.arange(B,device=DEVICE), target_phase].sum()
            grad = torch.autograd.grad(log_prob, x_in)[0]

            # ----- U-Net noise prediction ---------------------------------
            with torch.no_grad():
                noise_pred = unet(torch.cat([x, cond, prog_map, phase_map],1), t).sample

            sigma_t = noise_scheduler.sigmas[int(t)]
            guided_noise = noise_pred - guidance_scale * sigma_t * grad
            x = dpm_solver.step(guided_noise, t, x).prev_sample.detach()  # detach to avoid grad build-up
        ema.restore(unet.parameters())
        return x.clamp(-1,1)
```

```
    EPOCHS = 1000
    best_lpips = 1e9
    for epoch in range(1, EPOCHS+1):
        unet.train()
        pbar = tqdm(dl_train, desc=f"E{epoch}", leave=False)
        epoch_loss = 0.0
        val_loss   = 0.0
        val_seen   = 0
        for batch in pbar:
            cond   = batch["cond"].to(DEVICE)
            tgt    = batch["target"].to(DEVICE)
            prog   = batch["progress"].to(DEVICE)
            p_idx  = batch["phase_tgt_idx"].to(DEVICE)

            t = torch.randint(0, noise_scheduler.config.num_train_timesteps, (cond.size(0),), device=DEVICE).long()
            noise = torch.randn_like(tgt)
            noisy = noise_scheduler.add_noise(tgt, noise, t)

            B,_,H,W = cond.shape
            prog_map  = prog.view(B,1,1,1).expand(-1,-1,H,W)
            phase_map = one_hot_phase(p_idx, H,W)

            with torch.amp.autocast(device_type='cuda', dtype=torch.float16):
                model_in = torch.cat([noisy, cond, prog_map, phase_map],1)
                pred     = unet(model_in, t).sample
                loss     = F.mse_loss(pred, noise)
            scaler.scale(loss).backward()
            scaler.step(optimizer); scaler.update(); optimizer.zero_grad(set_to_none=True)
            ema.step(unet.parameters())
            epoch_loss += loss.item() * cond.size(0)
            pbar.set_postfix({"loss": f"{loss.item():.4f}"})
        print(f"[E{epoch}]  train MSE = {avg_loss:.4f}")

        # ----- fast-ish validation (8 batches) ---------------------------------
        unet.eval()
        ssim_v, lpips_v = [], []
        with torch.inference_mode(), torch.amp.autocast(device_type='cuda', dtype=torch.float16):
             pbar_val = tqdm(itertools.islice(dl_val, 8),
                             total=8, desc=f"[V{epoch}]", leave=False)
             for vb in pbar_val:
                c   = vb["cond"].to(DEVICE)
                p_i = vb["phase_tgt_idx"].to(DEVICE)
                prog = vb["progress"].to(DEVICE)
                out  = sample_phase(c, prog, p_i, steps=10)
                tgt  = vb["target"].to(DEVICE)
                ssim_v.append(ssim_batch(out, tgt).item())
                lpips_v.append(lpips_loss(out, tgt).mean().item())
                val_loss += F.mse_loss(out, tgt).item()*c.size(0)
                val_seen += c.size(0)
        ssim_m, lpips_m = np.mean(ssim_v), np.mean(lpips_v)
        avg_loss = epoch_loss / len(ds_train)
        avg_val  = val_loss   / val_seen
        print(f"[V{epoch}]  SSIM {ssim_m:.3f} | LPIPS {lpips_m:.3f} | val MSE = {avg_val:.4f}")

        # -------- comparison grid ----------------------------------------
        with torch.no_grad():
```

```
            vb = next(iter(dl_val))
            c      = vb["cond"].to(DEVICE)[:4]                 # 4 samples → 12 tiles
            tgt    = vb["target"].to(DEVICE)[:4]
            p_idx  = vb["phase_tgt_idx"].to(DEVICE)[:4]
            prog   = vb["progress"].to(DEVICE)[:4]

            gen = sample_phase(c, prog, p_idx, steps=20)        # or sample_phase_guided
            triplet = torch.cat([c, gen, tgt], 0)                       # [12,1,H,W]
            grid = vutils.make_grid(triplet, nrow=4,
                                    normalize=True, value_range=(-1,1), pad_value=1)
            # ---------- add a 24-pixel label band underneath --------------------
            grid_pil = T.ToPILImage()(grid.cpu()).convert("RGB")
            w, h = grid_pil.size
            label_h = 24
            canvas = Image.new("RGB", (w, h + label_h), (255, 255, 255))  # white strip
            canvas.paste(grid_pil, (0, 0))
            draw = ImageDraw.Draw(canvas)
            font = ImageFont.load_default()         # Pillow builtin – no extra installs
            for i, ph in enumerate(p_idx.cpu().tolist()):
                text = IDX2PHASE[ph]
                try:                                        # Pillow ≥ 8.0
                    bx0, by0, bx1, by1 = draw.textbbox((0, 0), text, font=font)
                    tw, th = bx1 - bx0, by1 - by0
                except AttributeError:                      # Pillow < 8.0 fallback
                    tw, th = font.getsize(text)
                x = i * (UNET_SIZE + 2) + (UNET_SIZE - tw) // 2         # centre in tile
                y = h + (label_h - th) // 2
                draw.text((x, y), text, fill=(0, 0, 0), font=font)
            canvas.save(f"E{epoch:04d}_grid.png")

        # ----- save best LPIPS model ---------------------------------------
        if lpips_m < best_lpips - 1e-4:
            best_lpips = lpips_m
            torch.save({"epoch":epoch,
                        "unet":unet.state_dict(),
                        "ema" :ema.state_dict(),
                        "optim":optimizer.state_dict()},
                       MODEL_DIR/"guided_diffusion_best.pth")
            print("↳ new *best* model saved")
```

```
[E1]  train MSE = 0.0268

[V1]  SSIM 0.067 | LPIPS 1.344 | val MSE = 1.0939
↳ new *best* model saved

[E2]  train MSE = 0.0276

[V2]  SSIM 0.119 | LPIPS 1.324 | val MSE = 0.8843
↳ new *best* model saved

[E3]  train MSE = 0.0161

[V3]  SSIM 0.152 | LPIPS 1.257 | val MSE = 0.7532
↳ new *best* model saved

[E4]  train MSE = 0.0131

[V4]  SSIM 0.149 | LPIPS 1.154 | val MSE = 0.8239
↳ new *best* model saved

[E5]  train MSE = 0.0110
```